CENG 336 Computer Networks

Real-World Network Behavior Profiling using Wireshark

Abdullah Ghassan Hussain Alamarah

230204900

Introduction

Using Wireshark this report shows the result of a network traffic analysis on the basis of three experiments: HTTPS vs HTTP browsing, audio streaming vs file downloading, and Wi-Fi vs mobile hotspot. The experiments were designed to compare real-world protocol behavior across different network conditions, and the analysis focused on protocol behavior at the DNS, TCP, TLS, and QUIC layers. All captures were done on a MacBook Pro using Wireshark on the Adnan Ötüken İl Halk Kütüphanesi library network and a personal mobile hotspot.

Experiment Design

Experiment 1: HTTPS vs HTTP Browsing

Two captures were done for this experiment on the library Wi-Fi. Browsing HTTPS sites (google.com, Wikipedia.org, monkeytype.com), and HTTP sites (httpforever.com, neverssl.com)

My hypothesis HTTPS will be encrypted at the application layer, with only IP of some sites visible. HTTP traffic though will expose everything – requests, response content including headers and server information, and page HTML in plain text – DNS queries will be visible on both cases though regardless of encryption.

Experiment 2: Audio Streaming vs File Download

Spotify Web was used for audio streaming on the university library Wi-Fi for approximately 3 minutes. A 100MB test file was downloaded from speed.hetzner.de on the same network.

My hypothesis is that Streaming will generate continuous small UDP/QUIC packets to a fixed server IP, while file downloading will generate a high-volume sustained TCP stream with larger packet sizes. The download is expected to produce higher retransmission rates due to network congestion caused by sustained high throughput.

Experiment 3: Wi-Fi vs Mobile Hotspot

Setup: Spotify Web streaming was repeated under identical conditions on a personal mobile hotspot to compare against the library Wi-Fi capture.

My hypothesis is that The mobile hotspot will show different latency characteristics and potentially higher retransmission rates due to the shared and variable nature of cellular connectivity compared to a wired university network infrastructure.

Methodology

A. Experiment Design

We set up three experiments to see how the network behaves when things are different. For each experiment we made sure to record what happened for least three minutes. We also kept a log of what we did. When we did it for each session. We used Wireshark 4.6.5 on a MacBook Pro that was connected to the internet using the en0 Wi-Fi interface to do all the captures.

B. Packet Capture

We did five capture sessions. Saved them as raw PCAP files. Here is what we got:

- capture1_https.pcap was from browsing the internet using HTTPS on the library Wi-Fi. It was 27 MB
- capture2_http.pcap was from browsing the internet using HTTP on the library Wi-Fi and it was 6.6 MB
- capture3_spotify.pcap was from listening to music on Spotify on the library Wi-Fi and it was 8.5 MB
- capture4_download.pcap was from downloading a 100MB file on the library Wi-Fi and it was 115 MB
- capture5_hotspot_spotify.pcap was from listening to music on Spotify using a hotspot

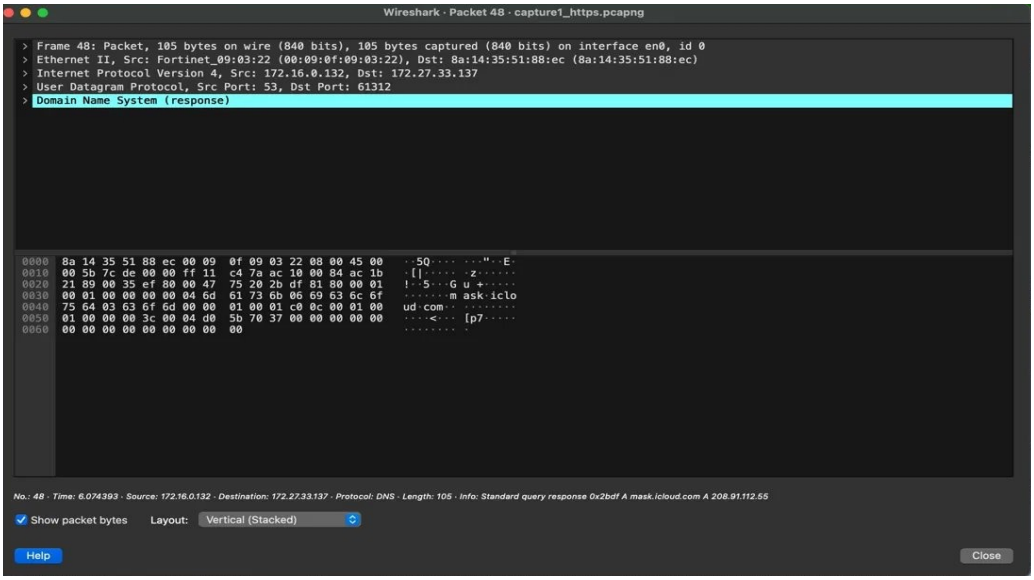
All of the captures came from the same place, which is 172.27.33.137 on the en0 interface. We did not filter anything while we were capturing the traffic. We did use filters when we looked at the data.

C. Action Logs

We kept a log of what we did. When we did it for each capture session. This log had things like when we went to a page when we played or paused music when we skipped a song and when we started a download. We used these logs to see how what we did on the internet was related to what happened with the packets, in the -layer analysis section. We looked at the network behavior. The action logs for the network behavior and the action logs. The action logs helped us understand the network behavior and the network behavior helped us understand the action logs. The network behavior was important. So were the action logs.

DNS Analysis

DNS queries were looked at from capture1_https.pcap. A filter was used to show DNS queries that were not responses which is dns && dns.flags.response == 0. This filter revealed 182 DNS queries out of 28,007 packets. Here are the findings:



Query Types and Domains

The capture revealed a mix of user-initiated and background DNS queries. User-initiated queries included domains such as www.google.com, www.wikipedia.org, and www.github.com, corresponding directly to browsing activity. A significant volume of background queries was also observed from services running without direct user interaction, including:

- mask.icloud.com — Apple iCloud Private Relay, queried repeatedly throughout the session
- googleleads.g.doubleclick.net, googlesyndication.com — Google advertising trackers loaded by visited pages
- eu-office.events.data.microsoft.com — Microsoft telemetry
- ws.audioscrobbler.com — Last.fm scrobbling service running in the background
- gateway.fe2.apple-dns.net — Apple DNS gateway

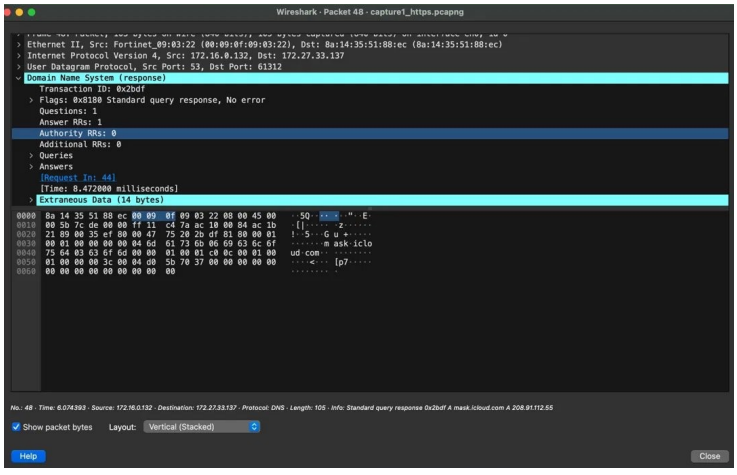
This demonstrates that even during a simple browsing session, a large portion of DNS traffic is generated by background services and third-party trackers rather than direct user actions.

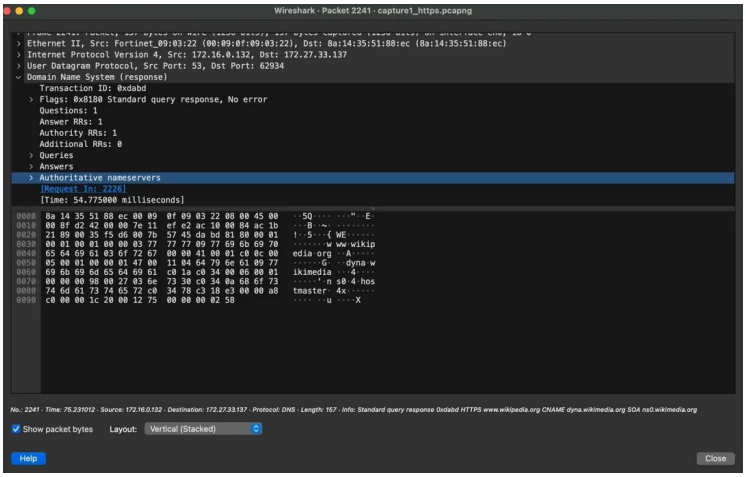
DNS Response Times

Three DNS response times were measured by examining individual response packets:

Domain	Response Time
mask.icloud.com	8.472 ms
googleleads.g.doubleclick.net	32.078 ms
www.wikipedia.org	54.775 ms

The variation in response times reflects differences in caching behavior at the DNS server level. The mask.icloud.com domain is queried so frequently that it is likely cached at the local DNS resolver, resulting in a significantly faster response. The www.wikipedia.org response was slower and returned an Authority RR with a CNAME record pointing to dyna.wikimedia.org, indicating a more complex resolution chain.



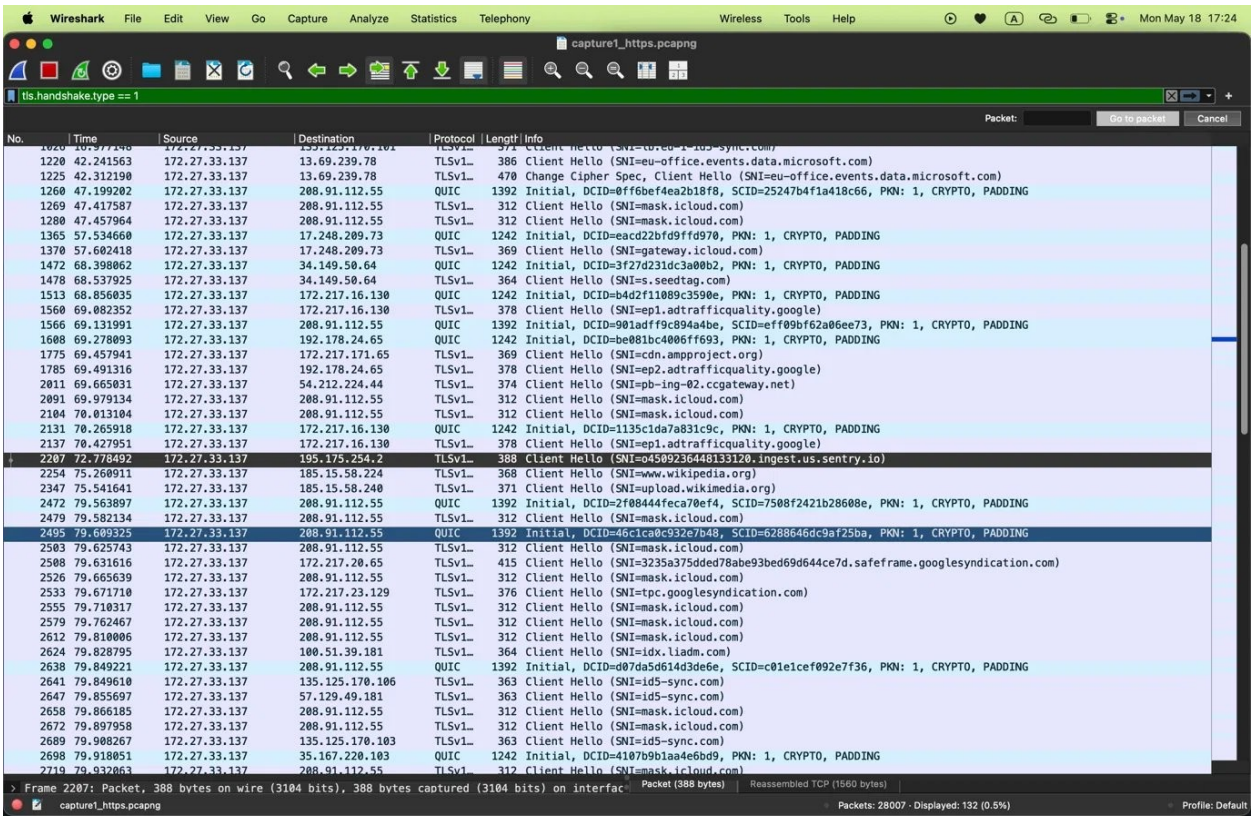


Caching Behavior

The website mask.icloud.com is checked a lot during the session. This is because the DNS TTL for mask.icloud.com is very short. So even if we visit the site a lot we still have to look it up every time. This makes sense because Apples Private Relay changes the IP address a lot to keep things private.

TLS Analysis

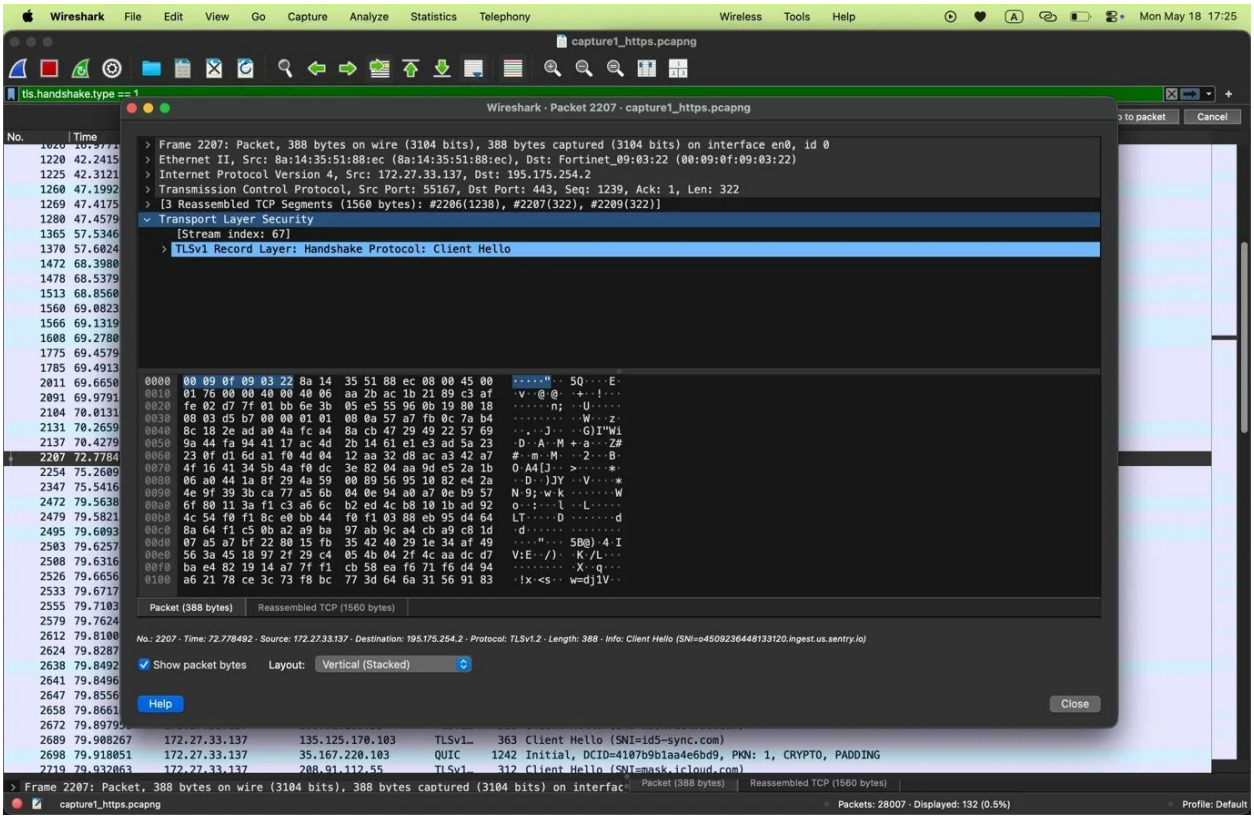
We looked at the capture1_https.pcap file. Found some TLS handshake packets. We used a filter to find these packets. It showed us 132 Client Hello packets out of 28,007 total packets. All of the TLS traffic went to port 443. This is what we found when we looked at the TLS traffic for mask.icloud.com and other sites. The TLS traffic, for these sites always goes to port 443.



Handshake Mechanism

Each TLS connection begins with a Client Hello packet in which the client advertises its supported cipher suites and extensions to the server. Examination of packet 2207 revealed the following handshake details:

- Session ID Length: 32 — indicating the client is attempting to resume a previous TLS session, reducing handshake overhead



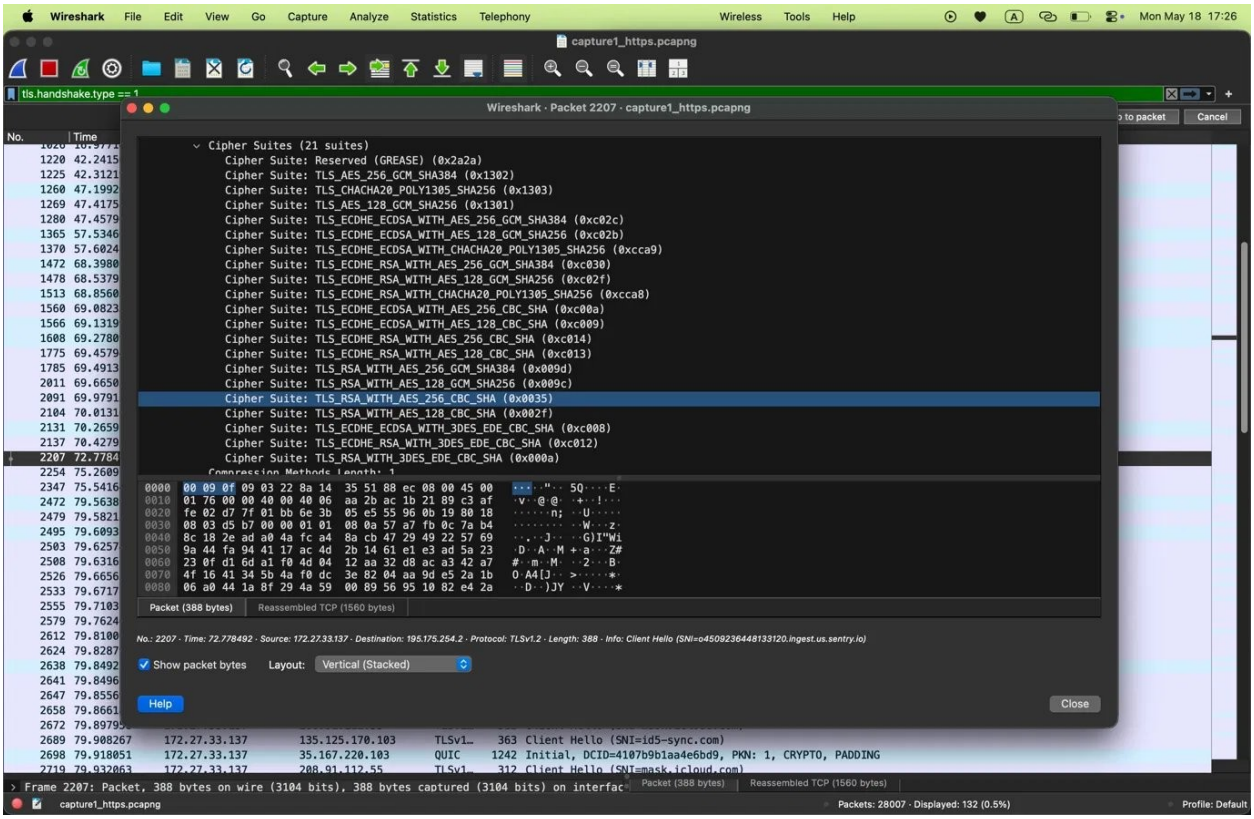
- Cipher Suites: 21 suites offered — ranging from modern TLS 1.3 suites to legacy suites for backwards compatibility
- Extension: server_name (SNI) — the destination hostname is transmitted in plaintext before encryption is established, meaning the domain being visited is visible to network observers even over HTTPS

Cipher Suites

The 21 cipher suites offered by the client ranged from strongest to weakest:

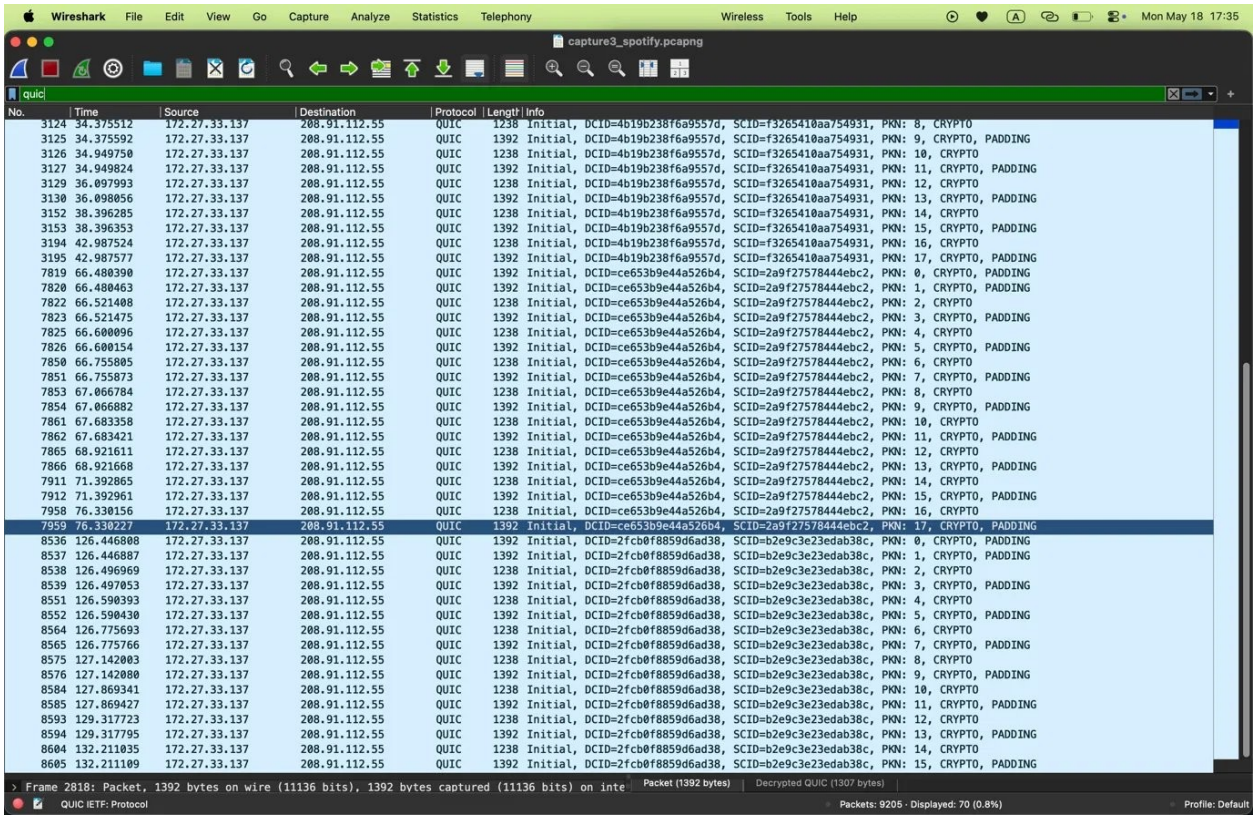
- TLS 1.3 suites (strongest): TLS_AES_256_GCM_SHA384, TLS_CHACHA20_POLY1305_SHA256, TLS_AES_128_GCM_SHA256
- TLS 1.2 suites: TLS_ECDHE_RSA_WITH_AES_256_GCM_SHA384, TLS_ECDHE_ECDSA_WITH_CHACHA20_POLY1305_SHA256
- Legacy suites (weakest): TLS_RSA_WITH_3DES_EDE_CBC_SHA, TLS_ECDHE_RSA_WITH_3DES_EDE_CBC_SHA

The presence of legacy 3DES cipher suites at the bottom of the list indicates the client maintains backwards compatibility with older servers, though in practice the server will negotiate the strongest mutually supported suite.



TLS Versions Observed

TLS Versions Seen, we saw both TLS 1.2 and TLS 1.3 connections. The protocol column had packets marked as TLSv1.2 going to systems like Microsoft and Sentry.io., Newer services used TLS 1.3.



QUIC Analysis

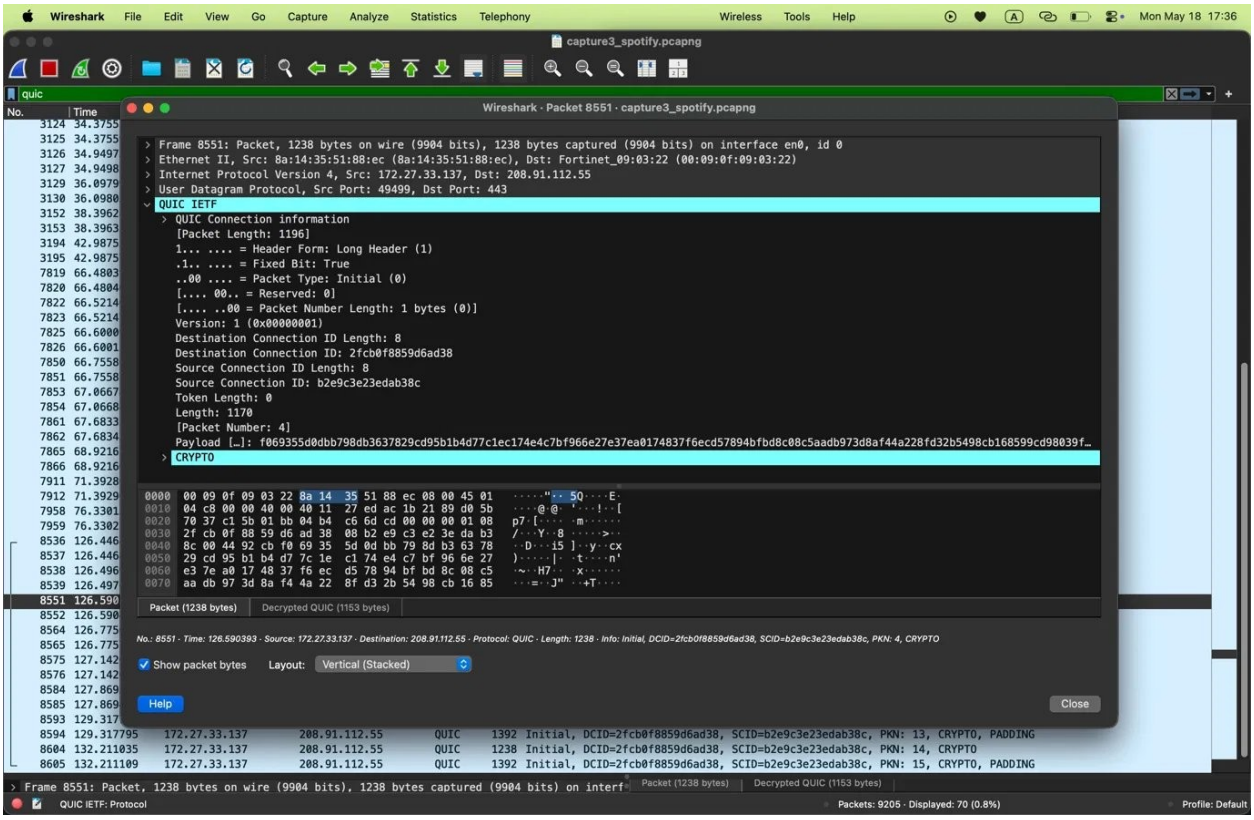
We found QUIC traffic in two captures: capture1_https.pcap and capture3_spotify.pcap.

In the Spotify capture we filtered by quick. Found 70 QUIC packets out of 9,205 total packets.

All these packets went to Spotifys server at 208.91.112.55, on UDP port 443.

Examination of a QUIC Initial packet revealed:

- QUIC Version: 1 (0x00000001) — the current IETF standard



- Header Form: Long Header — used during the handshake phase
- Packet Type: Initial — connection establishment
- Transport: UDP port 443 — unlike TLS which runs over TCP, QUIC operates over UDP
- Payload: fully encrypted — including most header fields

TLS vs QUIC Comparison

Feature	TLS over TCP	QUIC
Transport layer	TCP	UDP
Port	443	443
Handshake	TCP handshake + TLS handshake (2 round trips)	Combined in 1 round trip
Packet loss	Handled by TCP retransmission	Handled internally by QUIC
Header encryption	Partial	Most fields encrypted
Observed in	capture1_https.pcap	capture3_spotify.pcap

QUIC's advantage over TLS in streaming scenarios is clear — by combining the transport and security handshakes into a single round trip and running over UDP, Spotify achieves lower connection latency and avoids TCP's head-of-line blocking problem where a single lost packet stalls the entire stream.

Deep Packet Analysis: TCP Behavior Analysis

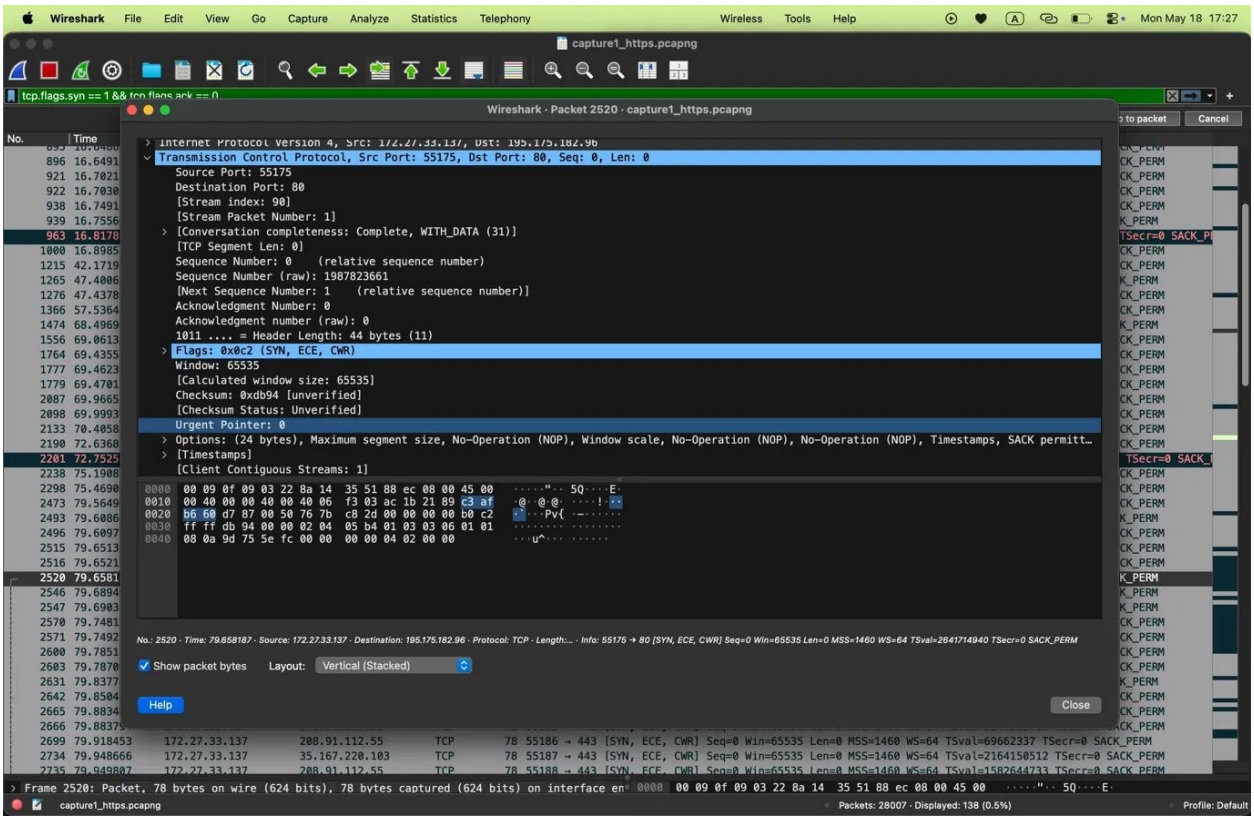
The Three-Way Handshake Process

In the connections analyzed in the capture1_https.pcap file we used a filter to look at packets that had a SYN flag set to 1 and an ACK flag set to 0 which revealed 138 SYN packets. These packets were initiating TCP connections throughout the whole session.

Lets take a look at one of these SYN packets, packet 2520. Here are the details:

- Source Port: 55175 → Destination Port: 80
- Sequence Number: 0
- Flags: SYN, ECE, CWR. These flags tell us that Explicit Congestion Notification or ECN is enabled.
- Initial Window Size: 65535 bytes
- Maximum Segment Size or MSS: 1460 bytes. That is pretty standard for Ethernet.
- Window Scale Factor: 64. This means that the maximum window size is actually 65535 × 64. That comes out to, around 4MB.

The ECE and CWR flags let us know that the client supports ECN. ECN is a way for routers to signal congestion before packets are lost. This allows for adjustment of data transfer.



Window Size Evolution

The window size is 65535 bytes. This is the size that can be received when a connection starts.

During the analysis of retransmitted packets, we found one packet with a window size of 2051.

When we scaled it the window size became 131264 bytes.

This decreases from the size shows that TCPs congestion control mechanism reduced the sending rate.

It responded to network congestion, which's consistent with the high number of retransmitted packets, on the library network.

Retransmissions

We looked at retransmission behavior in all captures using a filter. Here are the results:

In one of our captures named capture1_https.pcap we saw TCP Fast Retransmission events.

Capture	Total Packets	Retransmissions	Rate
HTTPS browsing (Wi-Fi)	28,007	473	1.70%
File download (Wi-Fi)	95,974	454	0.47%
Spotify hotspot	10,501	53	0.50%

These events came from the server 146.75.118.248, which's a Fastly CDN node.

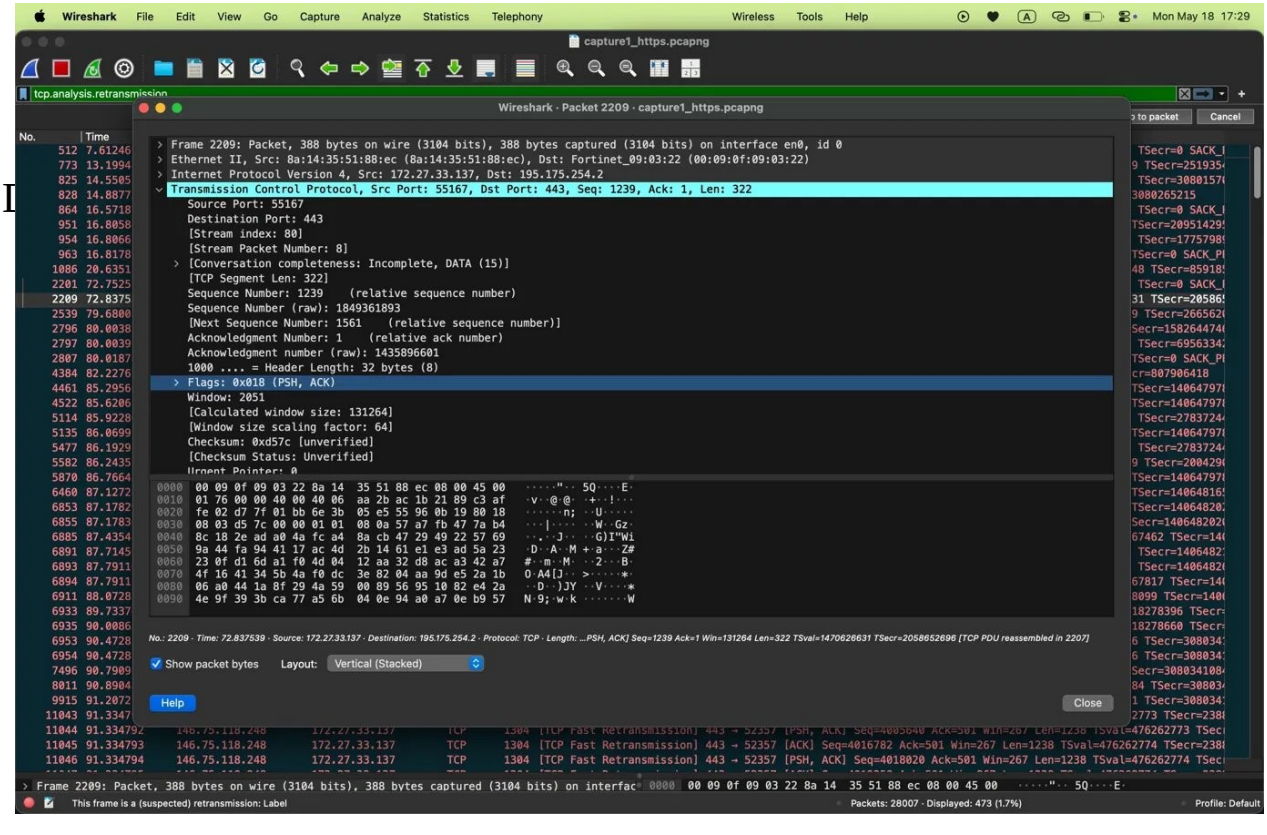
Fast Retransmission happens when the sender gets three duplicate ACKs before a timer expires.

This shows that the receiver found a gap in sequence numbers.

It is more efficient, than waiting for a timeout.

This also tells us that the network had some packet loss. It was not completely congested.

I



Anomaly 1: Elevated Retransmission Rate on Library Wi-Fi

The HTTPS browsing capture, which is the capture1_https.pcap file had a lot of problems with sending information. This happened 1.70 percent of the time which is much higher than what is usually okay for a stable wired network. Normally this number should be between 0.1 and 0.5 percent. We found a total of 473 packets that had to be sent. Many of these packets were sent from the Fastly CDN server with the address 146.75.118.248 using a method called TCP Fast Retransmissions.

This problem is because of the network environment. The Ankara Bilim Üniversitesi library network is used by a lot of people at the time. When we were capturing the data many users were sending information over the network, which caused much data to be sent at the same time. This made the network gateway get too full and drop some packets, which then had to be sent. We can tell this happened because when packets had to be sent the TCP window size got much smaller. It started at 65,535 bytes. Went down to as low as 2,051 bytes. This means that the network was slowing down the connection on purpose because it detected that some packets were lost.

What is interesting is that this problem did not happen the way in all of the captures. When we looked at the file download capture it only had a 0.47 percent retransmission rate even though it was sending a lot data. This suggests that the network can handle one stream of data better than many short streams of data that are sent at the same time like when people are browsing the web.

The HTTPS browsing capture which is the capture1_https.pcap file had a lot of problems with sending information. This happened 1.70 percent of the time which is much higher than what is usually okay for a stable wired network. Normally this number should be between 0.1 and 0.5 percent. We found a total of 473 packets that had to be sent. Many of these packets were sent from the Fastly CDN server with the address 146.75.118.248 using a method called TCP Fast Retransmissions.

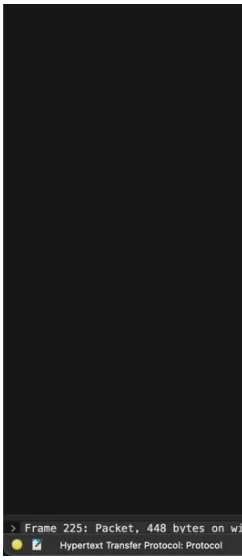
This problem is because of the network environment. The Ankara Bilim Üniversitesi library network is used by a lot of people at the time. When we were capturing the data many users were sending information over the network, which caused much data to be sent at the same time. This made the network gateway get too full and drop some packets, which then had to be sent. We can tell this happened because when packets had to be sent the TCP window size got much smaller. It started at 65,535 bytes. Went down to as low as 2,051 bytes. This means that the network was slowing down the connection on purpose because it detected that some packets were lost.

What is interesting is that this problem did not happen the way in all of the captures. When we looked at the file download capture it only had a 0.47 percent retransmission rate even though it was sending a lot data. This suggests that the network can handle one stream of data better, than many short streams of data that are sent at the same time like when people are browsing the web using the HTTPS browsing capture, which is the capture1_https.pcap file.

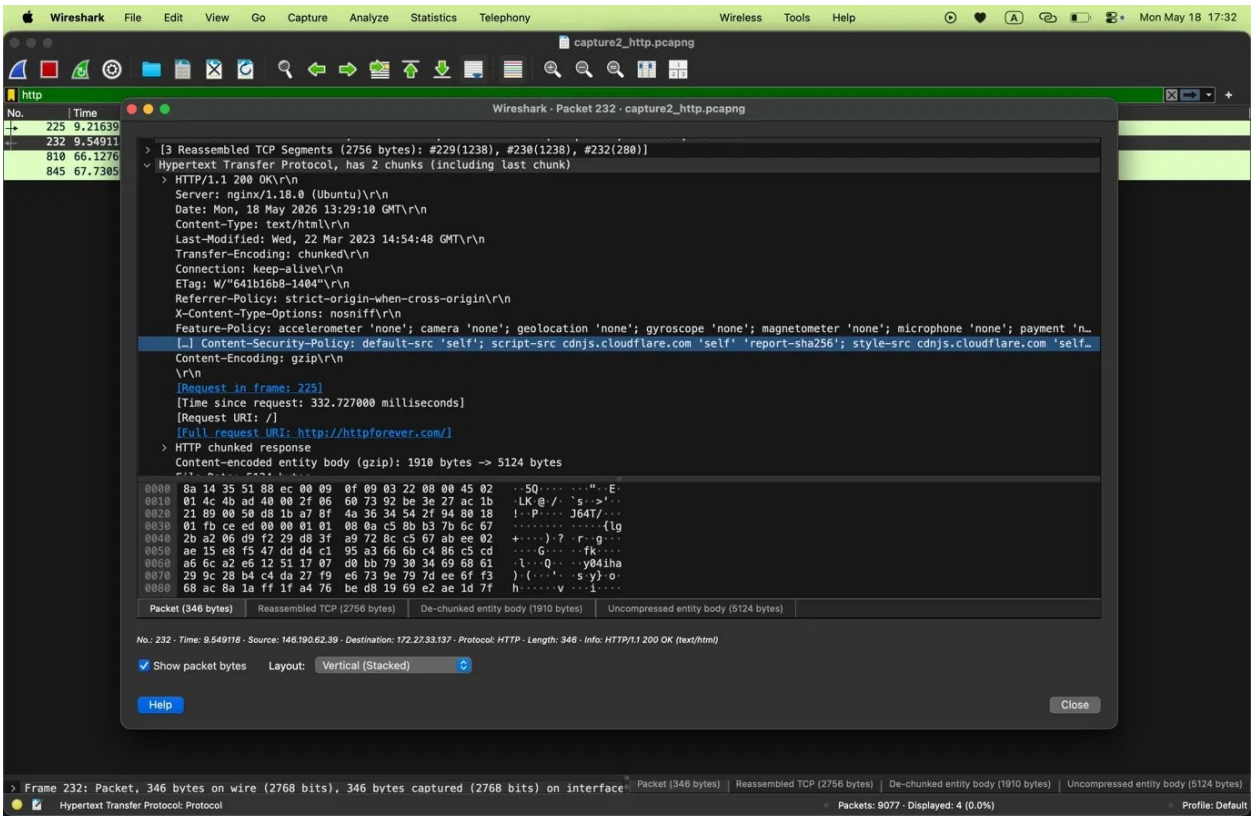
Anomaly 2: HTTP Traffic Present in HTTPS Capture

During analysis of capture2_http.pcap, only 4 HTTP packets were observed out of 9,077 total packets (0.04%). While the capture was intentionally designed to browse HTTP sites, the overwhelming

majority of traffic remained HTTPS or encrypted, even from sites visited during the session.



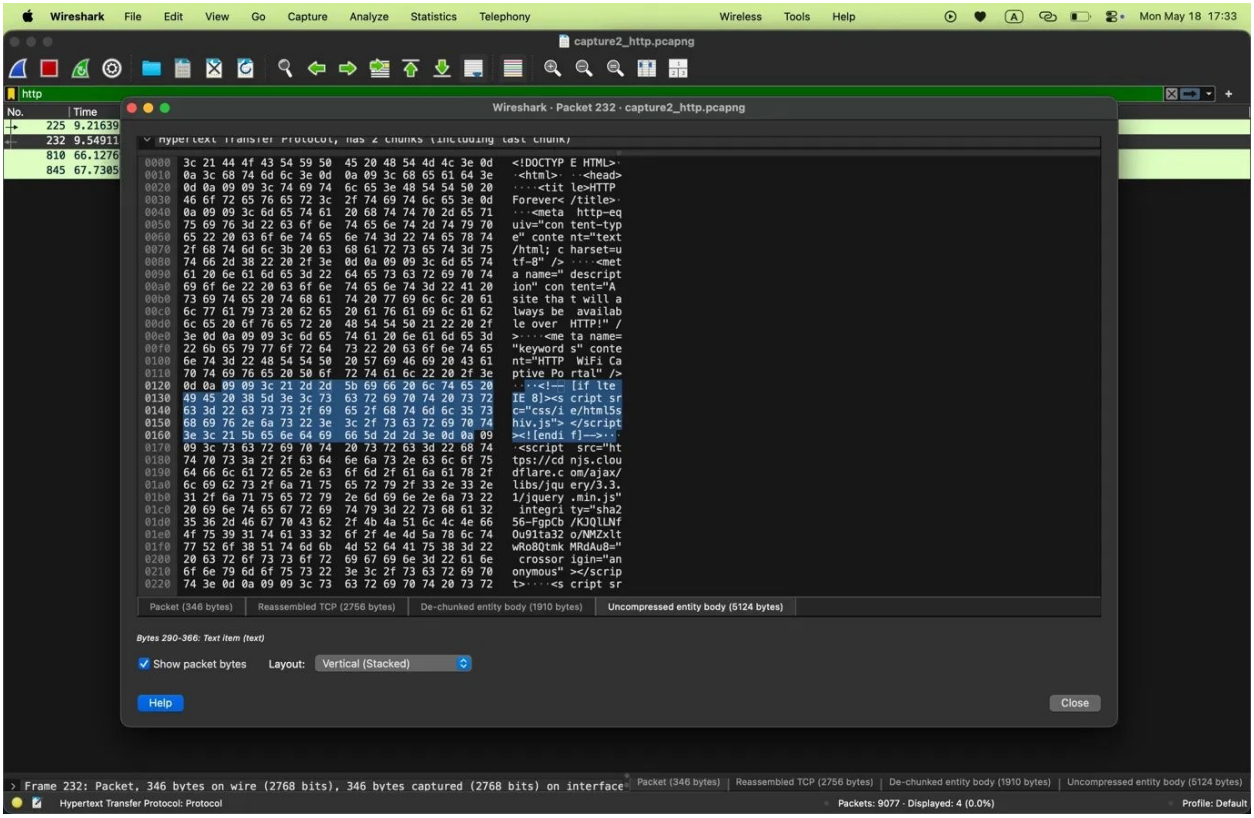
More significantly, examination of the HTTP response packet (packet 232) from httpforever.com revealed complete exposure of sensitive connection metadata in plaintext, including:



- Server software and version: nginx/1.18.0 running on Ubuntu
- Full request URI: http://httpforever.com/
- Exact response timestamp: Mon, 18 May 2026 13:29:10 GMT
- Complete HTML page source code visible in the uncompressed entity body

This anomaly highlights a critical security vulnerability of HTTP — any observer on the same network, including the library network administrator or a malicious actor performing a man-in-the-middle attack, could capture and read the complete content of unencrypted web sessions. In contrast, the HTTPS capture revealed none of this information, with only the destination IP and SNI hostname visible before encryption was established.

The near-complete absence of HTTP traffic in modern captures confirms that the web has largely transitioned to HTTPS, making plain HTTP increasingly rare and its use a significant security risk when it does occur.



Cross-Layer Correlation

Cross-layer correlation relates observed application-level user actions to their corresponding transport and network-layer behavior in the packet captures. Three correlations are presented below based on the timestamped action logs and packet-level evidence.

Correlation 1: HTTPS Browsing — DNS to TLS to Data Transfer

When a new website was visited during the HTTPS browsing session, a consistent sequence of events was observed across layers:

1. DNS layer: A query was sent from 172.27.33.137 to the local DNS resolver at 172.16.0.132, resolving the destination hostname. For example, opening www.wikipedia.org triggered a DNS query that returned after 54.775ms with a CNAME record pointing to dyna.wikimedia.org.
2. Transport layer: Immediately following DNS resolution, a TCP SYN packet was sent to the resolved IP address on port 443, initiating the three-way handshake with an initial window size of 65,535 bytes and ECN flags set.
3. Security layer: A TLS Client Hello was sent advertising 21 cipher suites and including the SNI extension with the destination hostname in plaintext, allowing the server to present the correct certificate.
4. Application layer: Following successful TLS negotiation, all application data was encrypted and unreadable, confirming HTTPS was functioning correctly.

This four-step sequence — DNS → TCP handshake → TLS handshake → encrypted data — was consistently observed for every new HTTPS connection initiated during the session, demonstrating the complete protocol stack in action.

Correlation 2: Spotify Streaming — User Actions to QUIC Traffic Patterns

During the Spotify streaming capture, user actions produced visible changes in network traffic patterns:

- Pressing play: Triggered an immediate burst of QUIC Initial packets to Spotify's server at 208.91.112.55 on UDP port 443, with packet numbers starting from 0 and incrementing sequentially. This represents the QUIC connection establishment and initial audio buffer fill.
- Continuous playback: Sustained a steady flow of QUIC packets at regular intervals, reflecting Spotify's adaptive bitrate streaming delivering audio chunks continuously to maintain playback.
- Skipping a song: Produced a brief interruption in the QUIC stream followed by a new burst of packets as the new track's audio data was fetched and buffered.

This demonstrates QUIC's efficiency for streaming — connection establishment and data transfer happen in fewer round trips than TLS over TCP would require, minimizing the audible gap when switching tracks.

Correlation 3: Wi-Fi vs Hotspot — Network Conditions to Retransmission Behavior

Comparing the Spotify streaming capture on library Wi-Fi (capture3_spotify.pcap) with the identical experiment on mobile hotspot (capture5_hotspot_spotify.pcap) revealed an unexpected result:

Metric	Library Wi-Fi	Mobile Hotspot

Total packets	9,205	10,501
Retransmissions	—	53
Retransmission rate	—	0.50%

The hotspot capture produced a comparable retransmission rate to the Wi-Fi download capture (0.47%), and both were significantly lower than the HTTPS browsing session on the same library Wi-Fi (1.70%). This suggests that at the time of capture, the library network was under heavier congestion from simultaneous users than the mobile hotspot, contrary to the initial hypothesis that cellular connectivity would be less stable.

At the application layer, no difference in Spotify playback quality was observed between the two network conditions, demonstrating QUIC's resilience — its built-in packet loss recovery and connection migration capabilities absorbed the network differences transparently without interrupting audio streaming.

Reflection

Differences Across Networks

The three network conditions captured in this project — university library Wi-Fi during browsing, university library Wi-Fi during download, and mobile hotspot during streaming — produced measurably different traffic characteristics despite all being standard internet connections.

The most significant difference was the retransmission rate on the library Wi-Fi during HTTPS browsing (1.70%) compared to both the file download on the same network (0.47%) and the mobile hotspot (0.50%). This reveals that network quality is not simply a property of the connection type but is heavily influenced by the number of simultaneous users and the nature of the traffic being generated. A shared university network serving hundreds of simultaneous users creates competition for bandwidth that manifests at the packet level as increased retransmissions and reduced TCP window sizes.

The mobile hotspot, despite being a cellular connection which is generally assumed to be less stable than a wired university network, performed comparably or better during the streaming session. This challenges the common assumption that institutional networks are inherently more reliable than mobile connections and highlights the importance of empirical measurement over theoretical assumptions.

Visibility Under Encryption

One of the findings of this project is about what you can still see even when traffic is encrypted.

Even though HTTPS hides all the content the following information is still visible in the capture1_https.pcap file:

- Destination IP addresses of every server you contact

- The exact domain names you visit are still visible in the TLS Client Hello packets as SNI hostnames
- Every domain you resolve is visible including background tracking and telemetry services through DNS queries
- When you visit sites and how long pages take to load's revealed through traffic timing and volume patterns
- Connection details like TCP window sizes, retransmission events and packet sizes are also visible

This shows that encryption protects what you are looking at but not information.

Someone watching on the library network can create a profile of your browsing behavior just from DNS queries and IP addresses without decoding a single packet.

This finding has implications for your privacy.

To be truly anonymous you need HTTPS and also DNS encryption like DoH/DoT and traffic obfuscation tools.

In contrast the HTTP capture is the case, It shows everything:

- page content
- server software
- timestamps
- full URIs

all in text, Anyone, on the network can read it.

Inference of User Behavior from Traffic Patterns

Even without decrypting any traffic, user behavior could be inferred from packet-level observations alone:

- DNS query timing revealed the sequence and approximate timing of website visits during the HTTPS browsing session
- QUIC packet bursts in the Spotify capture clearly identified music streaming activity, song skip events, and playback pauses through changes in traffic volume and timing
- Sustained high-volume TCP streams in the download capture unambiguously indicated a large file transfer
- Background DNS queries to domains such as ws.audioscrobbler.com, mask.icloud.com, and eu-office.events.data.microsoft.com revealed which applications were running on the device even without any deliberate user action

This behavioral fingerprinting is possible entirely from metadata, confirming that network-level privacy requires more than application-layer encryption. Protocol-level measures such as QUIC's encrypted headers, Apple's iCloud Private Relay (observed repeatedly in the captures), and DNS-over-HTTPS are increasingly important countermeasures against traffic analysis.

Capture 1 – HTTPS - WIFI

16:20:00 – started capture

16:20:10 – opened monkeytype.com

16:21:15 – opened Wikipedia.com

16:22:10 – opened apple.com.tr

16:23:10 – stopped capture

Capture 2 – HTTP – WIFI

16:29:00 – started capture

16:29:10 – opened httpforever.com

16:30:10 – opened neverssl.com

16:31:10 – opened google.com

16:32:10 – stopped capture

Capture 3 – Spotify streaming – WIFI

16:46:00 – started capture

16:46:05 – Pressed play on a playlist

16:47:00 – Pressed skip

16:48:00 – Pressed stop/pause

16:48:30 – Pressed play/resume

16:49:00 – stopped capture

Capture 4 – Download file – WIFI

16:57:00 – started capture

16:57:05 – opened speed.hetzner.com

16:57:40 – started downloading 100 MB Bin

16:58:25 – download finished

16:59:00 – stopped capture

Capture 5 – Spotify streaming – hotspot

17:07:00 – started capture

17:07:10 – Pressed play on a playlist

17:08:10 – pressed skip

17:09:00 – pressed stop/pause

17:09:30 – pressed play/resume

17:10:00 – stopped capture